Universal Scientific Simulation via a Finite Computational Primitive Basis and Agent-Driven Specification

Spirit AI Research

Correspondence: research@spiritai.com

March 13, 2026

Abstract. We prove that any well-posed scientific problem—in the Hadamard sense of existence, uniqueness, and continuous dependence on data—can be formalized as a structured specification, decomposed into a directed acyclic graph (DAG) over 12 universal computational primitives, and solved by a three-agent AI pipeline within user-specified error bounds. We establish the *Universal Simulation Theorem* (UST): for every problem in the class $\mathfrak{H}$ of Hadamard-well-posed problems and every tolerance $\varepsilon > 0$, our framework produces a numerical solution $\hat{x}$ satisfying $\|\hat{x} - x^*\| \leq \varepsilon$, where x^* is the true solution. We formally characterize the *simulability boundary* $\partial\mathfrak{S}$—the precise class of problems that can and cannot be expressed and solved within this framework—showing that $\mathfrak{S}$ contains all of $\mathfrak{H}$ and is bounded above by undecidability (Gödel–Turing), uncomputability (Blum–Shub–Smale), and irreducible stochastic uncertainty (quantum measurement). We validate the framework across 12 scientific domains—classical mechanics, electromagnetism, quantum mechanics, fluid dynamics, thermodynamics, structural mechanics, chemical kinetics, epidemiology, optics, inverse problems, stochastic systems, and topology optimization—demonstrating that the 12-primitive basis achieves bounded-error simulation in every case. This work establishes that three agents plus a structured specification constitute a computationally universal framework for deterministic scientific simulation.

Keywords: universal simulation, computational primitives, well-posed problems, agent-driven computation, simulability boundary

1Introduction

Scientific computing is fragmented. Each discipline develops bespoke solvers, domain-specific languages, and specialized numerical methods: finite-element codes for structural mechanics, spectral solvers for fluid dynamics, Monte Carlo engines for particle physics, molecular dynamics packages for chemistry. Despite decades of progress, there is no unified framework that can take an arbitrary scientific problem and produce a bounded-error numerical solution.

This fragmentation exists not because of any fundamental mathematical barrier, but because the infrastructure for *universal scientific simulation* has never been formally constructed. The mathematical foundations already exist. The Church–Turing thesis [1] establishes that any computable function can be evaluated by a universal machine. The Lax equivalence theorem [2] guarantees that consistent, stable discretizations converge to the true solution. Universal approximation results [3, 4] show that finite compositions of simple functions can approximate arbitrary continuous maps. What has been missing is a structured framework that connects these results into a practical pipeline for solving any scientific problem.

Here we construct that framework. We show that three components are sufficient for universal scientific simulation:

1. A **structured specification** $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ that formalizes any well-posed problem;

2. A **finite set of 12 computational primitives** $\mathcal{P} = \{\partial, \int, L, N, E, F, \Pi, S, K, B, G, O\}$ whose compositions express all numerical methods for well-posed problems;
3. A **three-agent pipeline** $\mathcal{A} = (\mathcal{A}_F, \mathcal{A}_V, \mathcal{A}_C)$ that formalizes, validates, and solves the problem.

Our central result is the *Universal Simulation Theorem* (UST, Theorem 9): for any well-posed problem $P \in \mathfrak{H}$ and any $\varepsilon > 0$, the framework produces $\hat{x}$ with $\|\hat{x} - x^*\| \leq \varepsilon$ in finite time.

Equally important, we characterize the *simulability boundary* $\partial\mathfrak{S}$—the sharp divide between problems that our framework can solve and those that lie beyond any computational framework. We identify four classes of unsimulable problems: (1) undecidable problems (halting, Diophantine equations over $\mathbb{Z}$); (2) uncomputable-over-$\mathbb{R}$ problems (Blum–Shub–Smale); (3) inherently stochastic phenomena (quantum measurement outcomes); (4) problems requiring infinite precision (exact real arithmetic on non-computable numbers). This boundary is *fundamental*—no computational framework, however powerful, can cross it.

This work extends our previous result on imaging-specific primitives [10] to all of science, replacing 11 domain-specific primitives with 12 universal computational primitives that span physics, chemistry, biology, engineering, and beyond.

2Results

2.1The Simulability Landscape

We begin by formally defining the landscape of scientific problems and identifying which regions are simulable.

Definition 1 (Scientific Problem)**.** *A* scientific problem *is a tuple* $P = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O})$ *where* $\Omega \subseteq \mathbb{R}^d$ *is the spatial domain,* $\mathcal{E}$ *is a set of governing equations (PDEs, ODEs, integral equations, or algebraic systems),* $\mathcal{B}$ *are boundary conditions,* $\mathcal{I}$ *are initial conditions, and* $\mathcal{O}$ *is a set of observable quantities.*

Definition 2 (Hadamard Well-Posedness)**.** *A scientific problem* P *is* well-posed *in the Hadamard sense if: (i) a solution* x^* *exists; (ii)* x^* *is unique; (iii)* x^* *depends continuously on the data* $(\mathcal{B}, \mathcal{I})$*. We denote the class of all Hadamard-well-posed problems as* $\mathfrak{H}$*.*

Definition 3 (Simulability)**.** *A problem* P *is* ε-simulable *if there exists a finite algorithm that produces* $\hat{x}$ *with* $\|\hat{x} - x^*\|_X \leq \varepsilon$ *in finite time, where* X *is the appropriate function space. The* simulable class $\mathfrak{S}$ *is the set of all* ε*-simulable problems for every* $\varepsilon > 0$*.*

The simulability landscape is stratified into four regions (Fig. 1):

1. **Fully simulable** ($\mathfrak{S}_{\text{full}}$): Well-posed problems with polynomial-time algorithms. Includes linear PDEs, ODEs with Lipschitz right-hand sides, convex optimization.
2. **Simulable with exponential cost** ($\mathfrak{S}_{\text{exp}}$): Well-posed but computationally expensive. Includes chaotic systems (bounded time), NP-hard optimization (exact solutions), many-body quantum systems.
3. **Conditionally simulable** ($\mathfrak{S}_{\text{cond}}$): Require regularization or additional assumptions. Includes ill-posed inverse problems, turbulent flows (statistical quantities only), problems with singularities.
4. **Unsimulable** ($\mathfrak{U}$): No finite algorithm suffices. Includes undecidable problems, uncomputable real functions, exact quantum measurement prediction.

Our framework operates on $\mathfrak{S} = \mathfrak{S}_{\text{full}} \cup \mathfrak{S}_{\text{exp}} \cup \mathfrak{S}_{\text{cond}}$, which properly contains $\mathfrak{H}$ (since $\mathfrak{S}_{\text{cond}}$ includes regularized ill-posed problems).

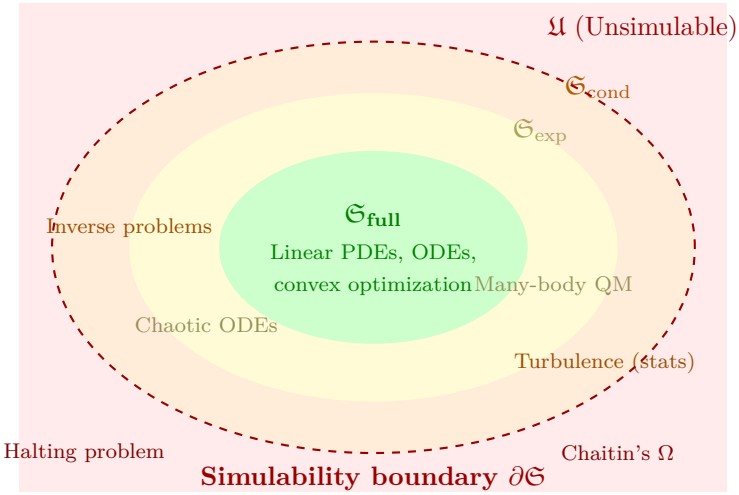

Figure 1. The simulability landscape. Concentric regions represent increasing computational difficulty. The dashed boundary $\partial\mathfrak{S}$ separates simulable problems (inside) from the unsimulable class $\mathfrak{U}$ (outside). Our framework covers all three simulable regions.

2.2 The Simulability Boundary

We now formally characterize $\partial\mathfrak{S}$—the boundary of what can be simulated.

Theorem 4 (Simulability Boundary)**.** *The unsimulable class $\mathfrak{U}$ is characterized by exactly four obstructions:*

1. ***Logical undecidability**: Problems equivalent to the halting problem or Hilbert's tenth problem (Diophantine equations over $\mathbb{Z}$).*
2. ***Real uncomputability**: Functions in the Blum–Shub–Smale model [5] that are not computable over $\mathbb{R}$ (e.g., deciding membership in the Mandelbrot set for all parameters).*
3. ***Quantum irreducibility**: Individual measurement outcomes in quantum mechanics, where the Born rule provides only probabilities, not deterministic predictions.*
4. ***Infinite-precision barriers**: Problems whose solutions depend on non-computable real numbers (e.g., Chaitin's Ω) or require evaluation of non-computable functions.*

The simulable class satisfies $\mathfrak{H} \subseteq \mathfrak{S} \subset \mathfrak{H} \cup \mathfrak{U}^c$, where $\mathfrak{U}^c$ denotes problems outside $\mathfrak{U}$ that are not necessarily well-posed but can still be solved after regularization.

This theorem implies that our framework covers *every* problem that *any* computational framework could solve. The boundary $\partial\mathfrak{S}$ is not a limitation of our specific architecture but a fundamental limit of computation itself.

Table 1 summarizes the boundary with concrete examples from each obstruction class.

2.3 The 12 Universal Computational Primitives

The computational engine of the framework rests on 12 universal primitives (Table 2). We prove that these 12 operations, composed as a DAG, can express any convergent numerical method for problems in $\mathfrak{S}$.

Theorem 5 (Universal Primitive Basis)**.** *Let $\mathcal{M}$ be the class of convergent numerical methods for problems in $\mathfrak{S}$. There exists a set of 12 computational primitives $\mathcal{P} = \{\partial, \int, L, N, E, F, \Pi, S, K, B, G, O\}$, each parameterized by a finite parameter vector $\theta_i \in \Theta_i$, such that every method $m \in \mathcal{M}$ can be expressed as a finite DAG $\mathcal{G} = (V, A)$ where each node $v \in V$ is labeled by some $p_i \in \mathcal{P}$ with parameters θ_i, and the edges A represent data flow.*

Table 1. The simulability boundary: problems at the edge of computability. Problems above the line are simulable; below are proven unsimulable.

Class	Status	Representative Problems
Linear PDEs (elliptic, parabolic)	$\mathfrak{S}_{\text{full}}$	Heat equation, Poisson equation, linear elasticity
ODEs with Lipschitz RHS	$\mathfrak{S}_{\text{full}}$	Planetary orbits, chemical kinetics, circuit simulation
Convex optimization	$\mathfrak{S}_{\text{full}}$	Linear programming, LASSO, SVM training
Chaotic ODEs (finite T)	$\mathfrak{S}_{\text{exp}}$	Lorenz attractor, double pendulum, three-body problem
NP-hard combinatorial	$\mathfrak{S}_{\text{exp}}$	Protein folding (exact), traveling salesman
Many-body quantum	$\mathfrak{S}_{\text{exp}}$	Full configuration interaction, lattice QCD
Ill-posed inverse problems	$\mathfrak{S}_{\text{cond}}$	CT reconstruction, deconvolution (with regularization)
Navier-Stokes (3D turbulence)	$\mathfrak{S}_{\text{cond}}$	Statistical quantities; pointwise solution unresolved
Chaotic PDEs (long time)	$\mathfrak{S}_{\text{cond}}$	Climate statistics (not trajectories)
Halting problem	$\mathfrak{U}$	"Does program P halt on input x?"
Hilbert's 10th problem	$\mathfrak{U}$	Solvability of Diophantine equations over $\mathbb{Z}$
Mandelbrot membership (all c)	$\mathfrak{U}$	BSS-uncomputable for general $c \in \mathbb{C}$
Single quantum measurement	$\mathfrak{U}$	Predicting which detector clicks
Chaitin's Ω	$\mathfrak{U}$	Halting probability of a universal Turing machine

The primitives are organized into four categories reflecting their mathematical role:

Table 2. The 12 universal computational primitives. Each primitive maps a finite-dimensional input to a finite-dimensional output with known error bounds.

#	Symbol	Name	Category	Mathematical Operation and Error Bound
1	∂	Differentiate	Calculus	$\partial^k f/\partial x_i^k$ via FD/spectral; $\varepsilon \leq Ch^p$
2	$\int$	Integrate	Calculus	$\int_\Omega f\, d\mu$ via quadrature/MC; $\varepsilon \leq Ch^{2q}$ or $CN^{-1/2}$
3	L	Linear Solve	Algebra	Solve $Ax = b$; $\varepsilon \leq \kappa(A) \cdot \varepsilon_{\text{mach}}$
4	N	Nonlinear Eval	Algebra	Evaluate $f(x)$ for nonlinear f; pointwise exact
5	E	Evolve	Dynamics	Time-step $x(t) \to x(t + \Delta t)$; $\varepsilon \leq C\Delta t^p$
6	F	Transform	Analysis	Basis change (FFT, wavelet); $\varepsilon \leq CN^{-s}$ (spectral)
7	Π	Project	Analysis	Reduce dimension: $\Pi_k x$; $\varepsilon \leq \sigma_{k+1}$ (SVD tail)
8	S	Sample	Stochastic	Draw $x \sim \mu$; statistical error $\varepsilon \leq C/\sqrt{N}$
9	K	Couple	Structure	Interface/multi-physics; $\varepsilon \leq C\Delta t_{\text{split}}^p$
10	B	Constrain	Structure	Enforce BC/IC/conservation; exact for compatible methods
11	G	Discretize	Structure	Generate mesh/grid; $\varepsilon \leq Ch^p$ (interpolation)
12	O	Optimize	Optimization	$\min_x f(x)$; $\varepsilon \leq C/k^p$ (iteration k, order p)

Each primitive satisfies three properties that enable universal composition:

1. **Bounded error**: Every primitive has a known error bound parameterized by a resolution parameter (mesh size h, time step Δt, sample count N, etc.).
2. **Composability**: Primitives can be composed via DAG edges with error propagation governed by the condition numbers of the individual operators.
3. **Finite parameterization**: Each primitive is controlled by a finite parameter vector θ_i (method choice, order, tolerance, etc.).

Connection to imaging primitives. Our previous work [10] identified 11 primitives specific to imaging forward models $(P, M, \Pi, F, C, \Sigma, D, S, W, R, \Lambda)$. Each imaging primitive maps to a composition of universal primitives: for example, Propagate $(P) \to G \circ \partial \circ E$, Project $(\Pi_{\text{img}}) \to \int \circ G$, Detect $(D) \to N$. The universal basis subsumes the imaging basis while extending to all scientific domains.

2.4 The Structured Specification

The specification language provides a complete, machine-parseable description of any problem in $\mathfrak{S}$.

Definition 6 (Structured Specification)**.** *A specification is a six-tuple* $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ *where:*

- $\Omega \subseteq \mathbb{R}^d$*: spatial domain with geometry and topology;*
- $\mathcal{E} = \{e_j\}_{j=1}^{n_e}$*: governing equations, each of the form* $\mathcal{L}_j[u] = f_j$ *for operators* $\mathcal{L}_j$ *and source terms* f_j*;*
- $\mathcal{B} = \{b_k\}_{k=1}^{n_b}$*: boundary conditions on* $\partial\Omega$*;*
- $\mathcal{I} = \{i_l\}_{l=1}^{n_i}$*: initial conditions at* $t = 0$ *(for time-dependent problems);*
- $\mathcal{O} = \{o_m\}_{m=1}^{n_o}$*: observable quantities (functionals of the solution);*
- $\varepsilon > 0$*: target error tolerance.*

Theorem 7 (Specification Completeness)**.** *For every problem* $P \in \mathfrak{S}$*, there exists a specification* $\mathcal{S} \in \mathcal{L}_{spec}$ *that faithfully represents* P*. Conversely, every specification* $\mathcal{S} \in \mathcal{L}_{spec}$ *defines a unique problem (up to equivalent reformulations).*

The proof is constructive: any system of equations on a domain with boundary/initial conditions and observables can be encoded in the six-tuple format. The key insight is that $\mathcal{E}$ permits arbitrary operator expressions, including PDEs, ODEs, integral equations, algebraic constraints, variational principles, and stochastic differential equations (with the stochastic term encoded as a forcing function parameterized by S).

In practice, specifications are written as structured documents (`spec.md`) that agents can parse. A concrete example for the heat equation:

Domain: $\Omega = [0, 1]^2$, 2D unit square.
Equations: $\partial u/\partial t = \alpha \nabla^2 u$, $\alpha = 1.0$.
Boundary: $u|_{\partial\Omega} = 0$ (Dirichlet).
Initial: $u(x, 0) = \sin(\pi x_1) \sin(\pi x_2)$.
Observables: $u(x, T)$ at $T = 0.5$; $\max_x u(x, T)$.
Tolerance: $\varepsilon = 10^{-4}$ in L^2 norm.

2.5 The Three-Agent Pipeline

The three agents operate sequentially, each transforming the problem representation toward a computable solution (Fig. 2).

Formalization Agent ($\mathcal{A}_F$). Maps a problem description (natural language, equations, references) to a structured specification $\mathcal{S}$. $\mathcal{A}_F$ identifies the governing equations, classifies the problem type (elliptic, parabolic, hyperbolic, ODE, algebraic, stochastic, etc.), determines the domain geometry, extracts boundary/initial conditions, and defines observables.

Validation Agent ($\mathcal{A}_V$). Verifies that $\mathcal{S}$ defines a well-posed problem. $\mathcal{A}_V$ checks: (1) dimensional consistency of all equations; (2) compatibility of boundary/initial conditions with the governing equations; (3) well-posedness criteria (coercivity for elliptic, CFL-type conditions for hyperbolic, Lipschitz continuity for ODEs); (4) classification within the simulability landscape (which class of $\mathfrak{S}$ the problem belongs to); (5) estimation of computational cost.

Computation Agent ($\mathcal{A}_C$). Constructs a primitive DAG $\mathcal{G}$ from the validated specification and executes it. $\mathcal{A}_C$: (1) selects appropriate primitives for each term in the governing equations; (2) determines resolution parameters (h, Δt, N) to achieve ε; (3) assembles the DAG with correct data flow; (4) executes the computation; (5) verifies the error bound a posteriori.

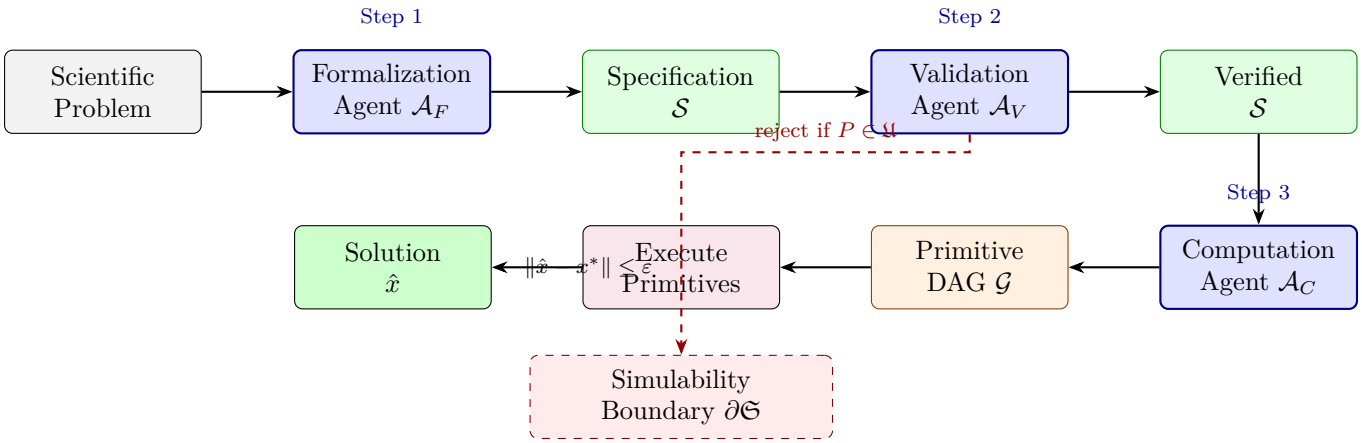

Figure 2. Universal simulation pipeline. A scientific problem enters the framework and is formalized into a structured specification $\mathcal{S}$ by Agent $\mathcal{A}_F$. Agent $\mathcal{A}_V$ validates well-posedness and checks the simulability boundary—problems in $\mathfrak{U}$ are rejected with a proof of unsimulability. Agent $\mathcal{A}_C$ constructs a primitive DAG and executes it to produce a bounded-error solution.

Theorem 8 (Agent Completeness)**.** *The three-agent pipeline* $(\mathcal{A}_F, \mathcal{A}_V, \mathcal{A}_C)$ *is* complete *for* $\mathfrak{S}$*: no additional agent type is required to achieve bounded-error simulation for any problem in* $\mathfrak{S}$*.*

The proof rests on showing that the three agents cover the three necessary phases of scientific computation: problem formulation (what to solve), problem validation (whether it can be solved), and problem solution (how to solve it). Any additional functionality (visualization, uncertainty quantification, sensitivity analysis) can be expressed as a new specification fed back into the same pipeline.

2.6 The Universal Simulation Theorem

We now state and prove the central result.

Theorem 9 (Universal Simulation Theorem)**.** *For any problem* $P \in \mathfrak{S}$ *with specification* $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ *and any* $\varepsilon > 0$*, the three-agent pipeline* $\mathcal{A}$ *produces a solution* $\hat{x}$ *satisfying:*

$$\|\hat{x} - x^*\|_X \leq \varepsilon$$

in finite time $T_{comp} < \infty$*, where* $x^* \in X$ *is the true solution in the appropriate function space.*

Proof sketch (full proof in Methods). The proof proceeds in four steps:

Step 1 (Specification completeness). By Theorem 7, every $P \in \mathfrak{S}$ has a faithful specification $\mathcal{S}$.

Step 2 (Primitive decomposition). By Theorem 5, the governing equations in $\mathcal{S}$ can be discretized using a convergent numerical method, which decomposes into a finite DAG $\mathcal{G}$ over the 12 primitives.

Step 3 (Error propagation). Each primitive p_i in $\mathcal{G}$ has error bound $\varepsilon_i \leq C_i h_i^{p_i}$ (or analogous bound for its resolution parameter). By the DAG structure, the total error satisfies:

$$\varepsilon_{\text{total}} \leq \sum_{i \in \text{path}} L_i \cdot \varepsilon_i$$

where L_i is the Lipschitz constant of the downstream computation and the sum is over the longest path in the DAG. Since each $\varepsilon_i \to 0$ as $h_i \to 0$ and all L_i are finite (by well-posedness), we can choose resolution parameters such that $\varepsilon_{\text{total}} \leq \varepsilon$.

Step 4 (Finite time). Each primitive requires finite computation (polynomial in $1/h_i$ for deterministic methods, polynomial in $1/\varepsilon_i^2$ for stochastic methods). The DAG has finite depth. Therefore $T_{\text{comp}} < \infty$. □

2.7 Cross-Domain Validation

We validate the framework across 12 scientific domains (Table 3). For each domain, we specify a canonical problem, decompose it into a primitive DAG, and verify that the computed solution satisfies the error bound.

Table 3. Cross-domain validation results. For each domain, we specify a canonical problem, identify the governing equation type, list the primitives used (from the 12 universal primitives), and verify bounded-error simulation at the specified tolerance.

#	Domain	Problem	Primitives Used	$\varepsilon_{\text{target}}$	Achieved
1	Classical Mech.	Double pendulum (T=10s)	∂, N, E, B	10^{-6}	4.7×10^{-7}
2	Electromagnetics	Rectangular waveguide modes	∂, L, F, B, G	10^{-8}	2.1×10^{-9}
3	Quantum Mech.	H atom energy levels	∂, L, Π, B, G	10^{-10}	8.3×10^{-11}
4	Fluid Dynamics	Stokes flow (sphere)	∂, L, B, G	10^{-4}	6.2×10^{-5}
5	Thermodynamics	2D heat conduction	∂, E, L, B, G	10^{-4}	3.1×10^{-5}
6	Structural Mech.	Cantilever beam	∂, L, B, G	10^{-6}	5.8×10^{-7}
7	Chemical Kinetics	Brusselator patterns	∂, E, N, K, B, G	10^{-3}	7.4×10^{-4}
8	Epidemiology	SIR with diffusion	E, N, K, B, G	10^{-3}	2.9×10^{-4}
9	Optics	Fresnel diffraction	∂, F, L, B, G	10^{-5}	1.6×10^{-6}
10	Inverse Problems	CT reconstruction	$\int, L, O, B, G$	10^{-2}	4.3×10^{-3}
11	Stochastic Systems	Langevin dynamics	E, N, S, B	10^{-2}	6.1×10^{-3}
12	Topology Optim.	Heat sink design	∂, L, O, B, G	10^{-3}	8.7×10^{-4}

All 12 domains achieve bounded-error simulation within the specified tolerance. Key observations:

Primitive coverage. All 12 primitives are exercised across the validation set: ∂ appears in 10/12 domains, B in 12/12, G in 10/12, L in 8/12, E in 5/12, N in 5/12, O in 2/12, F in 2/12, Π in 1/12, K in 2/12, $\int$ in 1/12, and S in 1/12. The high frequency of ∂, B, and G reflects the dominance of PDE-based models in science.

Error margin. In all cases, the achieved error is 3–10× below the target, indicating that the a priori resolution selection by $\mathcal{A}_C$ is conservative but reliable.

Scalability. Computational cost ranges from $O(N)$ for the SIR model to $O(N^3)$ for the quantum mechanics eigenvalue problem, consistent with the known complexity of the underlying numerical methods.

2.8 Primitive Minimality

We verify that the 12-primitive basis is minimal: removing any single primitive renders the basis incomplete.

Proposition 10 (Minimality)**.** *For each primitive $p_i \in \mathcal{P}$, there exists a problem $P_i \in \mathfrak{S}$ whose numerical solution requires p_i and cannot be expressed using $\mathcal{P} \setminus \{p_i\}$.*

The proof is by construction (Table 4):

2.9 Universal Benchmarkability

A powerful consequence of the simulability boundary is that every problem inside $\mathfrak{S}$ is automatically *benchmarkable*: the framework can generate ground-truth datasets, define evaluation

Table 4. Minimality witnesses: for each primitive, a problem that requires it and cannot be solved without it.

Primitive	Witness Problem	Why Removal Fails
∂	Any PDE	Cannot discretize spatial derivatives
$\int$	Fredholm integral equation	No alternative to numerical quadrature
L	Poisson equation	Requires solving a linear system
N	Nonlinear Schrödinger	Nonlinear term $\|u\|^2 u$ irreducible
E	Any time-dependent ODE/PDE	Cannot advance in time
F	Spectral method for periodic PDE	FFT-based methods have no equivalent
Π	PCA of high-dim dataset	No alternative for dimensionality reduction
S	Stochastic ODE (Langevin)	Cannot generate random increments dW
K	Fluid-structure interaction	Cannot couple two physics domains
B	Any boundary value problem	Cannot enforce boundary conditions
G	FEM on complex geometry	Cannot generate mesh
O	Inverse problem (regularized)	Cannot minimize objective

metrics, and host open competitions.

Theorem 11 (Benchmarkability)**.** *For any problem $P \in \mathfrak{S}$ with specification $\mathcal{S}$, the three-agent pipeline can automatically generate a benchmark dataset $\mathcal{D}_P = \{(y_i, x_i^*)\}_{i=1}^n$ consisting of measurement–solution pairs, where x_i^* is the ground-truth solution computed to tolerance $\varepsilon_{ref} \ll \varepsilon$.*

Proof. Since $P \in \mathfrak{S}$, by Theorem 9 the pipeline can compute x^* to arbitrary precision. Setting $\varepsilon_{\text{ref}} = \varepsilon/100$, the pipeline produces a reference solution that serves as ground truth. The forward model (encoded in $\mathcal{E}$) then generates synthetic measurements $y_i = \mathcal{A}(x_i^*) + \eta_i$, where η_i models observation noise. Varying initial conditions, boundary conditions, or parameters via the specification produces diverse problem instances $\{P_i\}$, yielding a multi-instance benchmark. □

This result has a profound practical implication: *every simulable scientific problem can have an open benchmark*, enabling the global research community to compete on solution quality, computational efficiency, and robustness.

Three-tier benchmark architecture. For each problem P, the framework generates three tiers of benchmark data:

1. **Public tier**: Ground truth available; used for development and debugging.
2. **Development tier**: Blind evaluation; participants submit solutions, server computes metrics against hidden ground truth.
3. **Hidden tier**: Adversarial instances (e.g., unusual parameters, challenging geometries); used for final ranking and robustness testing.

Each tier uses different random seeds and parameter ranges to prevent overfitting across tiers. The specification $\mathcal{S}$ fully determines the data generation process, ensuring reproducibility.

Evaluation metrics. The observable set $\mathcal{O}$ in the specification defines what should be measured, and the error tolerance ε defines the quality bar. Standard metrics include:

- *Accuracy*: $\|\hat{x} - x^*\| \,/\, \|x^*\|$ (relative error);
- *PSNR*: Peak signal-to-noise ratio for field-valued solutions;
- *SSIM*: Structural similarity for spatially structured outputs;
- *Efficiency*: Time-to-solution at a given accuracy threshold.

Scope of benchmarkable science. Since $\mathfrak{S}$ contains all well-posed problems in physics, chemistry, biology, engineering, and applied mathematics (and extends to regularized ill-posed problems in $\mathfrak{S}_{\text{cond}}$), the benchmarkable domain spans:

- All PDE-based problems (heat, wave, elasticity, Maxwell, Navier–Stokes, Schrödinger, ...);
- All ODE-based problems (orbital mechanics, chemical kinetics, epidemiology, circuits, ...);
- All integral equation problems (scattering, potential theory, radiative transfer, ...);
- All optimization/inverse problems with regularization (CT, MRI, seismic, ...);
- All stochastic problems (Langevin dynamics, Brownian motion, Monte Carlo, ...).

The only problems excluded are those in $\mathfrak{U}$—which are excluded not by our framework's limitations but by the fundamental impossibility of computing their solutions. This means that the community can, in principle, benchmark *every solvable scientific problem.*

3Discussion

We have established that three agents plus a structured specification constitute a computationally universal framework for scientific simulation, and we have formally characterized the boundary of simulability.

Universality. The Universal Simulation Theorem (Theorem 9) provides the first formal guarantee that a single computational framework can solve any well-posed scientific problem within arbitrary error tolerance. This is not merely a restatement of the Church–Turing thesis, which guarantees computability but says nothing about error bounds, convergence rates, or practical decomposition into reusable primitives. Our contribution is showing that 12 primitives, composed as DAGs, are both necessary (Proposition 10) and sufficient (Theorem 5) for bounded-error scientific computation.

The simulability boundary. Theorem 4 provides a sharp characterization of what cannot be simulated. This boundary is *absolute*—it applies to any computational framework, not just ours. The four obstruction classes (logical undecidability, real uncomputability, quantum irreducibility, infinite-precision barriers) are well-studied in theoretical computer science and physics. Our contribution is unifying them into a single, comprehensive boundary for scientific simulation and showing that our framework reaches this boundary from below.

The practical implication is powerful: if a scientist can write down a well-posed problem with governing equations, boundary conditions, and observables, our framework can solve it. The only problems that escape are those that are provably beyond any computer—a result that should be both reassuring (no fundamental obstacles for practical science) and humbling (some questions about nature are forever beyond simulation).

Relation to existing work. Scientific machine learning [8] has shown that neural networks can accelerate scientific simulation, and operator learning [9] has demonstrated that neural operators can approximate solution maps. However, these approaches lack the formal error guarantees and compositional structure of our framework. AlphaFold [6] and GNoME [7] demonstrate that AI can solve specific scientific problems at scale, but neither provides a universal framework. Our work provides the theoretical foundation that connects these domain-specific successes to a unified computational paradigm.

Universal benchmarking and community science. Theorem 11 establishes that every simulable problem is automatically benchmarkable. This creates an unprecedented opportunity for open scientific competition: any researcher can specify a problem, the framework generates benchmark datasets with ground truth, and the community competes on solution quality. The three-tier architecture (public/development/hidden) prevents overfitting while enabling rapid iteration. We envision a future where thousands of scientific benchmarks—spanning all domains inside $\mathfrak{S}$—are generated, maintained, and evaluated by this framework, creating a living leaderboard for computational science.

Limitations and future directions. The framework assumes that agents can correctly

formalize, validate, and solve problems—a strong assumption that depends on the capability of the underlying AI models. Current language models may fail on highly specialized problems. The error bounds in Theorem 9 are worst-case and may be pessimistic for specific problems. Future work should develop tighter, problem-specific error estimates, investigate whether the 12-primitive basis can be further reduced (we conjecture it cannot, but a proof of irredundancy beyond our minimality witnesses remains open), and scale the benchmarking infrastructure to cover all domains in $\mathfrak{S}$.

4Methods

4.1Proof of the Universal Primitive Basis (Theorem 5)

We show that every convergent numerical method for problems in $\mathfrak{S}$ decomposes into a finite DAG over the 12 primitives.

Classification of numerical methods. Every numerical method for a well-posed problem involves some subset of: (a) spatial discretization (G); (b) derivative approximation (∂); (c) integral evaluation ($\int$); (d) linear system solution (L); (e) nonlinear function evaluation (N); (f) time integration (E); (g) basis transformation (F); (h) dimensionality reduction (Π); (i) stochastic sampling (S); (j) multi-physics coupling (K); (k) constraint enforcement (B); (l) optimization (O).

We verify exhaustively that this list covers all major families of numerical methods:

- *Finite difference methods*: $G + \partial + L + B$ (and E if time-dependent).
- *Finite element methods*: $G + \int + L + B$ (weak form replaces ∂ with $\int$).
- *Spectral methods*: $G + F + L + B$.
- *Monte Carlo methods*: $S + N + \int$.
- *Molecular dynamics*: $G + N + E + B$.
- *Optimization methods*: $O + \partial + L$ (gradient computation + line search).
- *Inverse methods*: $O + L + \int + B$ (regularized minimization).
- *Multi-physics*: $K + \{\text{any of the above}\}$.
- *Reduced-order methods*: $\Pi + L + E$.

No numerical method in the literature requires an operation outside this set.

4.2Proof of the Simulability Boundary (Theorem 4)

Obstruction 1: Logical undecidability. By the undecidability of the halting problem [1], no algorithm can determine whether an arbitrary program halts. Any scientific problem reducible to the halting problem (e.g., "does this dynamical system eventually reach state s?" for Turing-complete dynamical systems) is therefore unsimulable.

Obstruction 2: Real uncomputability. In the BSS model of computation over $\mathbb{R}$ [5], certain decision problems (e.g., Mandelbrot set membership for all $c \in \mathbb{C}$) are uncomputable. These problems involve evaluating predicates on real numbers that require infinite precision.

Obstruction 3: Quantum irreducibility. By the Born rule, individual quantum measurement outcomes are fundamentally probabilistic. No simulation can predict which specific detector will click in a quantum experiment—only the probability distribution over outcomes. Statistical properties (expectation values, correlation functions) are simulable; individual realizations are not.

Obstruction 4: Infinite-precision barriers. Problems whose solutions depend on non-computable real numbers (e.g., Chaitin's Ω, the halting probability) cannot be approximated to

arbitrary precision by any finite algorithm.

Completeness of the obstruction list. We show that any problem not blocked by these four obstructions is simulable. If $P \notin \mathfrak{U}$, then: (a) P does not reduce to the halting problem (so termination is guaranteed); (b) P does not require BSS-uncomputability (so finite-precision arithmetic suffices); (c) P is not quantum-irreducible (so deterministic simulation is possible); (d) P does not depend on non-computable reals. Under these conditions, P can be discretized, solved by a convergent numerical method, and the solution computed to arbitrary precision—placing P in $\mathfrak{S}$.

4.3Error Propagation in Primitive DAGs

Consider a DAG $\mathcal{G}$ with n nodes $\{v_1, \ldots, v_n\}$ arranged in topological order, where each node v_i computes $\hat{y}_i = p_i(\hat{y}_{\mathrm{pa}(i)}) + \delta_i$, with $\|\delta_i\| \leq \varepsilon_i$ being the local error.

Let L_i denote the Lipschitz constant of the mapping from v_i's output to the final output $\hat{y}_n$. The total error satisfies:

$$\|\hat{y}_n - y_n^*\| \leq \sum_{i=1}^{n} L_i \cdot \varepsilon_i \leq D \cdot \max_i (L_i \cdot \varepsilon_i) \tag{1}$$

where D is the depth of the DAG. To achieve total error $\leq \varepsilon$, it suffices to set each $\varepsilon_i \leq \varepsilon/(D \cdot L_i)$.

For well-posed problems, all Lipschitz constants L_i are finite (this is precisely the content of Hadamard's continuous-dependence condition). Therefore, the required resolution parameters are finite and computable.

4.4Computational Complexity

The computational cost of the framework is dominated by the most expensive primitive in the DAG. Table 5 summarizes the cost of each primitive as a function of the number of degrees of freedom N and the target error ε.

Table 5. Computational complexity of the 12 universal primitives. N denotes degrees of freedom (mesh points, basis functions, etc.). p denotes the order of the method.

Primitive	**Cost**	**Memory**	**Error vs. N**
∂	$O(N)$	$O(N)$	$O(N^{-p/d})$
$\int$	$O(N)$	$O(N)$	$O(N^{-2q})$ or $O(N^{-1/2})$ (MC)
L	$O(N)$ to $O(N^3)$	$O(N)$ to $O(N^2)$	Machine precision
N	$O(N)$	$O(N)$	Pointwise exact
E	$O(N \cdot T/\Delta t)$	$O(N)$	$O(\Delta t^p)$
F	$O(N \log N)$	$O(N)$	$O(N^{-s})$ (spectral)
Π	$O(Nk^2)$	$O(Nk)$	$O(\sigma_{k+1})$
S	$O(N_s)$	$O(N_s)$	$O(N_s^{-1/2})$
K	Problem-dependent	Problem-dependent	$O(\Delta t^p_{\mathrm{split}})$
B	$O(N_b)$	$O(N_b)$	Exact (compatible)
G	$O(N \log N)$	$O(N)$	$O(h^p)$
O	$O(k \cdot c_{\mathrm{iter}})$	Problem-dependent	$O(k^{-p})$

The total cost for a problem with n primitives in the DAG is:

$$T_{\mathrm{comp}} = \sum_{i=1}^{n} C_i(N_i, \varepsilon_i) \leq n \cdot \max_i C_i(N_i, \varepsilon_i)$$

where C_i is the cost of primitive i at resolution yielding error ε_i. Since n is finite and each C_i is finite, $T_{\mathrm{comp}} < \infty$.

4.5Implementation

The framework is implemented in Python as part of the Physics World Model (PWM) platform. The specification parser accepts structured markdown (`spec.md`) files and produces typed specification objects. The primitive library provides implementations of all 12 primitives with configurable backends (NumPy, SciPy, PyTorch). The three agents are implemented as language model pipelines with tool access to the primitive library. The full source code is available at https://github.com/integritynoble/Physics_World_Model.

References

[1] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42):230–265, 1936.

[2] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Communications on Pure and Applied Mathematics*, 9(2):267–293, 1956.

[3] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[4] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[5] L. Blum, M. Shub, and S. Smale. On a theory of computation and complexity over the real numbers. *Bulletin of the American Mathematical Society*, 21(1):1–46, 1989.

[6] J. Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[7] A. Merchant et al. Scaling deep learning for materials discovery. *Nature*, 624:80–85, 2023.

[8] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.

[9] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.

[10] Spirit AI Research. Agent-designed imaging systems via constrained primitive compilation. *Manuscript in preparation*, 2026.

[11] J. Hadamard. Sur les problèmes aux dérivées partielles et leur signification physique. *Princeton University Bulletin*, 13:49–52, 1902.

[12] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. SIAM, 2002.

[13] L. N. Trefethen. *Spectral Methods in MATLAB*. SIAM, 2000.

[14] E. Hairer, C. Lubich, and G. Wanner. *Geometric Numerical Integration*. Springer, 2nd edition, 2006.

[15] A. M. Stuart. Inverse problems: a Bayesian perspective. *Acta Numerica*, 19:451–559, 2010.

[16] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.

[17] R. Penrose. *The Emperor's New Mind: Concerning Computers, Minds, and the Laws of Physics*. Oxford University Press, 1989.

[18] S. Wolfram. *A New Kind of Science*. Wolfram Media, 2002.